
"Những quyết định quan trọng, ảnh hưởng lâu dài, khó thay đổi"

- DB cho core data
- Chia module / service
- Xử lý trạng thái đơn hàng
- Payment gateway
- Quan sát lỗi

1.2.2. Sống trong thực tế

"Không chỉ để hệ thống chạy được, mà còn sống được"

Real user, tài, data và tiền, lỗi,

1.2.3. Phù hợp bối cảnh

"Không phải kiến trúc phức tạp nhất mà phù hợp nhất"

- App nội bộ 40 người khác Không hệ thống toàn cầu
- Team nhỏ => không hạ tầng lớn
- MVP cần tốc độ học
- Hệ thống lớn cần vận hành
- Lựa chọn ở mỗi giai đoạn

1.2.4. Trade-off

"Không có kiến trúc tối ưu mọi mặt. Mỗi lựa chọn đều có lợi ích và chi phí"

Monolith, Microservices, Cache, Queue

1.3. Kiến trúc không phải là gì

"Những thứ này có thể liên quan đến kiến trúc nhưng không tự động là kiến trúc tốt"

- Không chỉ là vẽ sơ đồ, chọn framework, NNLT, cloud, microservices

1.4. Kiến trúc trả lời gì?

"Một cuộc thảo luận kiến trúc tốt bắt đầu bằng câu hỏi đúng"

- Ai dùng?
- Phần lớn nào?

- Dữ liệu qua đâu?
 - Lỗi thì sao?
 - Tải tăng thì sao?
-

2. TKPM vs KTPM

2.1. Thiết kế vs kiến trúc

"Thiết kế làm code / module tốt, kiến trúc làm hệ thống tốt"

- Thiết kế: function, class, modul, pattern
- Kiến trúc: service, database, vận hành

2.2. Câu hỏi thuộc TKPM

"Cấp code, module, API nội bộ"

2.3. Câu hỏi thuộc KTPM

"Toàn hệ thống, vận hành, tiến hóa"

2.4. Trải nghiệm người dùng

"Người dùng không thấy db hay queue, nhưng cảm nhận trực tiếp hệ quản kiến trúc"

Nhanh chậm, đúng lỗi, mất không, thông báo không, lỗi ở đâu

2.5. Khả năng chịu tải

"App chạy tốt lúc ít người vẫn có thể sập vào giờ cao điểm"

2.6. Khả năng thay đổi

"Kiến trúc tốt => thay đổi có kiểm soát, không bị lan"

Coupling

2.7. Độ tin cậy

"Không phải hỏi có lỗi không, mà là phản ứng thế nào"

2.8. Bảo mật

""

Thuộc tính xuyên suốt

user => xác thực, phân quyền, mã hóa, mạng riêng, audit log, cảnh báo => db

2.9. Chi phí vận hành

"Kiến trúc ngẫu nhiên nhưng vượt quá năng lực và ngân sách => không tốt"

- Nhiều service = nhiều pipeline
 - Nhiều db = nhiều backup
 - Queue / cache = vận hành thêm
 - Microservices = tracing + observability
 - Cloud autoscaling = chi phí đột biến
-

3. Minicase công nghiệp

- Monolith không xấu, tùy giai đoạn